

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
if not os.path.exists('diabetes.csv'):
    !wget https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv
df = pd.read_csv('diabetes.csv')
fix_cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in fix_cols:
    df[col] = df[col].replace(0, df[col].median())
X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
imp = pd.Series(
    rf.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print("Feature Importances:")
print(imp.round(3))

imp_sorted = imp.sort_values()

colors = [
    'tomato' if v > imp.mean() else 'steelblue'
    for v in imp_sorted
]

imp_sorted.plot(
    kind='barh',
    color=colors,
    figsize=(8, 5),
    edgecolor='white'
)

plt.axvline(
    x=imp.mean(),
    color='black',
    linestyle='--',
    linewidth=1,
    label='Mean Importance'
)

plt.title('Random Forest Feature Importance - Diabetes Dataset')
plt.xlabel('Importance Score')
plt.legend()
plt.tight_layout()
plt.show()

top_features = imp[imp > imp.mean()].index.tolist()

print(f"\nTop Features: {top_features}")

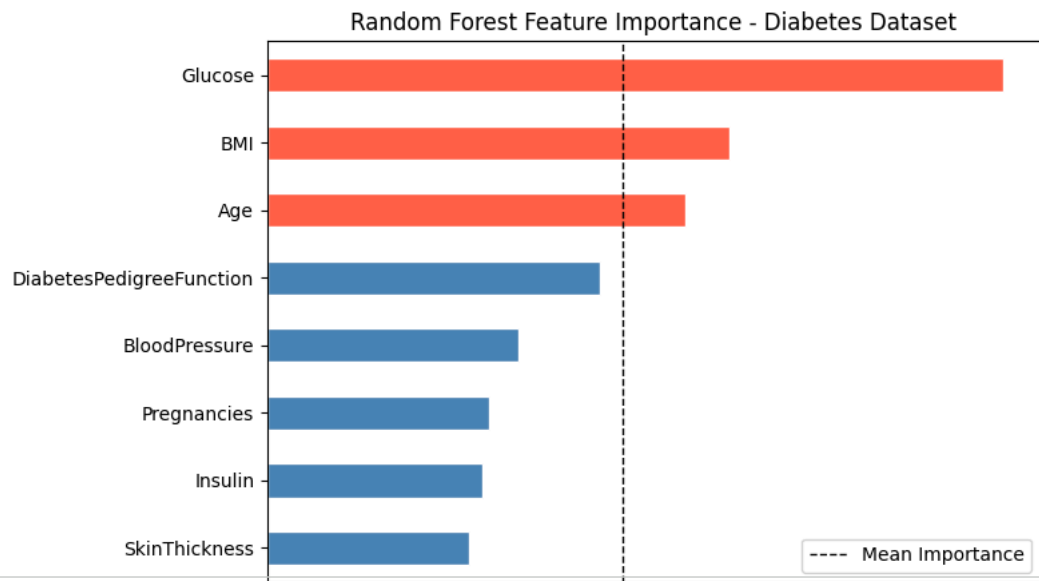
rf_top = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

rf_top.fit(X_train[top_features], y_train)

full_acc = rf.score(X_test, y_test)
top_acc = rf_top.score(X_test[top_features], y_test)
print(f"Accuracy with all {len(X.columns)} features : {full_acc:.2%}")
print(f"Accuracy with top {len(top_features)} features : {top_acc:.2%}")
print("Often comparable – low-importance features can add noise!")

```

```
Feature Importances:  
Glucose      0.259  
BMI          0.163  
Age          0.147  
DiabetesPedigreeFunction 0.117  
BloodPressure 0.088  
Pregnancies  0.078  
Insulin      0.076  
SkinThickness 0.071  
dtype: float64
```



Start coding or [generate](#) with AI.

```
Top Features: ['Glucose', 'BMI', 'Age']  
Accuracy with all 8 features : 77.27%  
Accuracy with top 3 features : 72.73%  
Often comparable – low-importance features can add noise!
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.